CONTROLLER AREA NETWORK

CAN Bus — Study Guide, Explanations & Conceptual Simulations

Built from the supplied 81-page CAN.pdf, with carefully separated explanatory notes and selected external technical references.

Guide scope	What is included
Source coverage	All 81 pages of the supplied CAN.pdf, in source order
Core concepts	Network architecture, addressing, masks/filters, physical layer, arbitration, frames, CRC, errors, controllers and CANoe
Visuals	Selected source figures, newly drawn explanatory diagrams and conceptual simulations
External references	Kvaser, Vector Informatik, Texas Instruments and other technical references; cited separately
Important boundary	This is a guide to the supplied lecture material, not a replacement for ISO 11898 or a complete implementation specification

Prepared: 14 November 2026

How to use this guide

Read the chapters in order. The lecture slides introduce the system visually, then revisit several concepts in more detail. This guide consolidates repeated slides while retaining the original page references.

Label	Meaning
PDF says	A fact, term, diagram or numerical value stated in the supplied CAN.pdf.
Explanation	A plain-language explanation added to make the PDF understandable. It does not change the source claim.
Conceptual simulation	A worked bit sequence or simplified model. It demonstrates logic; it is not a physical-layer simulator or conformance test.
External reference	A selected technical source used for a supporting visual or cross-check. It is not silently merged into the lecture content.

Source discipline: the supplied PDF is the primary authority for this guide. Where a slide is image-only, damaged, incomplete or ambiguous, the limitation is stated rather than filled with invented detail.

Source map

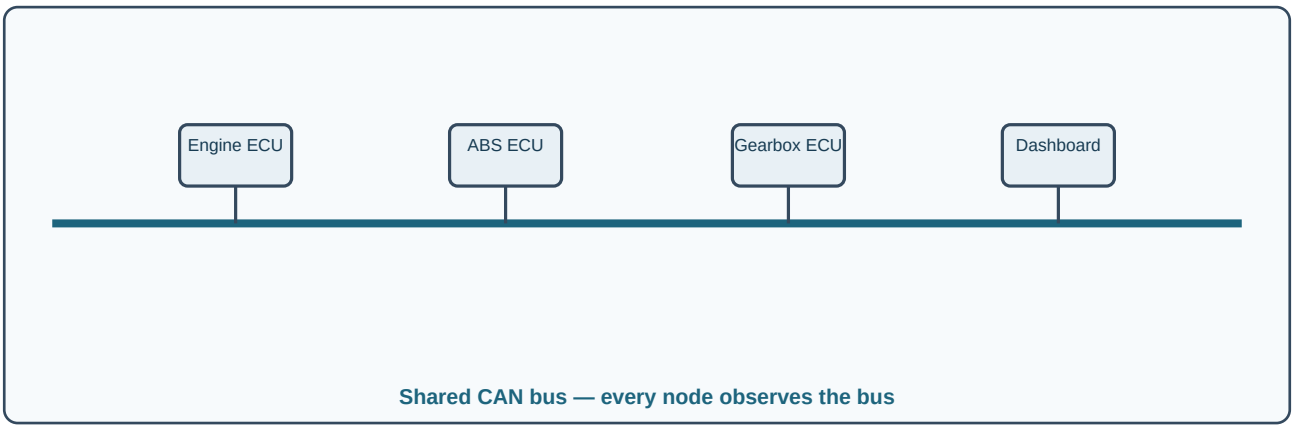
PDF pages	Main subject
1–12	Overview, development, bus concept, node configuration, physical errors and CAN overview
13–21	Addressing, broadcast model, mask and filter examples
22–29	NRZ, bit stuffing, differential signalling, transceiver, physical layer and EMI
30–40	CAN frame overview, wired-AND, arbitration, speed/length and priority
41–50	CAN protocol scope, higher-layer protocols, frame structure, CRC and bit stuffing
51–59	Standard/extended identifiers and four frame types
60–65	Fault prevention and data protection
66–81	Communication examples, controller filtering, CANoe, K-matrix and J1939 example

1. What CAN is and why it exists

1.1 The original problem — PDF pages 2–5

Modern vehicles contain many electronic control units (ECUs): engine management, ABS, transmission control, active suspension, lighting, air conditioning, airbags, central locking and other systems. These systems need to exchange information.

Before CAN, the systems were connected using many individual point-to-point wires. As the number of systems grew, the wiring harness became long and complicated. The PDF identifies higher material cost, longer production time and reduced reliability as consequences.



Conceptual redraw of the shared-bus arrangement shown on PDF pages 4–5. The original slides contrast dense point-to-point wiring with one shared serial bus.

CAN replaces many dedicated interconnections with a shared serial bus. Each control unit receives CAN-specific hardware that follows the communication protocol.

1.2 What the CAN system specifies — PDF page 2

Layer / function	What it means in this lecture
Physical layer	Electrical signalling and the bus medium.
Protocol layer	How messages are transmitted and received.
Message filtering layer	How a node accepts only messages relevant to it; the slide mentions add-on protocols in this context.

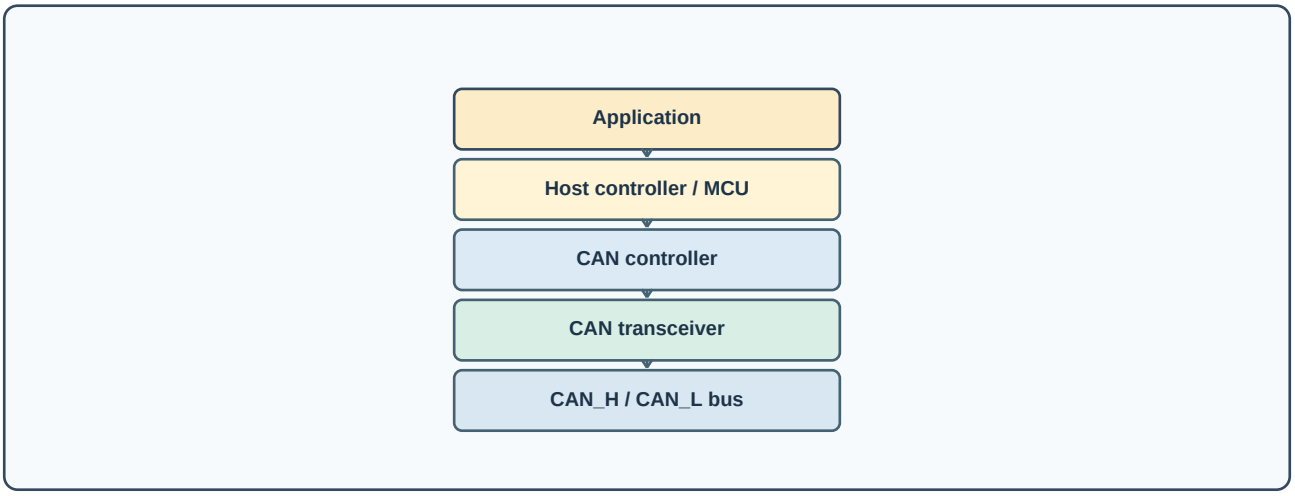
The lecture also highlights message prioritisation and the need to prevent small/low-cost nodes from being overloaded by irrelevant received messages.

1.3 CAN is broadcast — PDF page 6

Every node connected to the bus can observe every transmission. CAN does not send a frame to one exclusive physical destination. The receiving hardware applies local filtering; a node reacts only to frames that pass its acceptance rules.

Do not confuse broadcast reception with universal application processing. Every node can observe the bus, but acceptance filters decide which received identifiers are passed on for local processing.

1.4 Basic node architecture — PDF page 7



Redrawn functional stack from the Basic Configuration slide.

Block	Role
Application	The control function, e.g. ABS or engine management.
Host controller	The microcontroller/host that runs the application and exchanges data with the CAN controller.
CAN controller	Implements CAN protocol functions and manages transmit/receive messages.
CAN transceiver	Converts controller-side logic signals to/from the differential bus signals.
CAN bus	The shared CAN_H and CAN_L physical medium.

The slide shows example controller/host devices and 120 Ω termination at the two ends of the bus. The labels are examples from the lecture, not a requirement to use those specific chips.

2. Addressing and acceptance filtering

2.1 Two addressing ideas — PDF pages 13–14

The addressing slide contrasts peer-to-peer addressing with broadcast addressing. Peer-to-peer uses a destination/source header and a selected receiver. The slide says this can be used for diagnostics. Standard CAN is shown as broadcast: the receiver decides whether to receive or discard a frame.

Peer-to-peer (slide concept)	Broadcast (standard CAN concept in slide)
Sender selects a destination.	Sender puts an identifier in the frame.
Receiver is selected by destination.	All nodes observe the transmission.
The slide associates it with diagnostics.	The slide associates it with standard CAN.

The page 14 diagram shows a sender generating a data frame onto the shared bus. Each receiving node has a filter in its CAN-controller path.

2.2 Mask and filter logic — PDF pages 15–21

The mask determines which identifier bits matter during comparison. A mask bit of 1 means compare that bit. A mask bit of 0 means don't care. The filter supplies the expected value for the bits that are compared.

```
Acceptance rule (conceptual):
((received_ID XOR filter) AND mask) == 0
```

If the result is zero, every bit selected by the mask matches and the frame is accepted. Otherwise it is rejected.

Mask bit	Meaning
0	Ignore this identifier bit.
1	This identifier bit must match the filter bit.

2.3 The PDF's four mask/filter examples

PDF page	Desired receive set	Filter	Mask	Meaning
17	Only ID 0x00001567	0x00001567	0x1FFFFFFF	Every identifier bit must match.
18	0x00001560 through 0x0000156F	0x00001560	0x1FFFFFF0	Low four bits are ignored.
19	0x00001560 through 0x00001567	0x00001560	0x1FFFFFF8	Low three bits are ignored.
20	Any frame	0	0	All bits are don't-care.

Received ID	**0000156B**	Low 4 bits are ignored
Filter	**00001560**	0x156B is accepted
Mask	**1FFFFFF0**	0x1570 is rejected

Worked acceptance example: received ID 0x156B matches filter 0x1560 when the low four bits are ignored.

Example: 0x156B is within the 0x1560–0x156F range, so it is accepted by the page 18 rule. 0x1570 differs in a bit that is not masked out, so it is rejected.

2.4 Common mistake

A CAN identifier is not a destination address. It labels the message and participates in arbitration. The filter is a local receiver-side decision; it does not prevent other nodes from seeing the frame.

3. Bit encoding and the physical layer

3.1 NRZ — PDF page 22

NRZ means Non-Return-to-Zero. A bit is represented by a level that is held for the bit interval; the signal does not return to a neutral level between every bit. A transition is associated with a change in the transmitted bit value.

The PDF notes that fewer transitions on average reduce electromagnetic interference, but the receiver must tolerate oscillator drift. Bit stuffing is introduced to help with synchronization.

3.2 Bit stuffing — PDF pages 22 and 50

The PDF states: after five consecutive bits of equal value, the transmitter inserts one bit of the opposite value. The receiver discards that inserted bit. A sixth consecutive bit of the same value is treated as an error.

Original run	Transmitted on bus	Receiver action
11111	111110	Remove the inserted 0.
00000	000001	Remove the inserted 1.
111111	Not a valid stuffed sequence	Stuffing error condition.

The PDF's page 50 diagram marks the stuffed region: from the start of frame through the CRC sequence. Do not apply the stuffing rule indiscriminately to every field; the slide itself shows a bounded region.

3.3 Bit-stuffing simulation

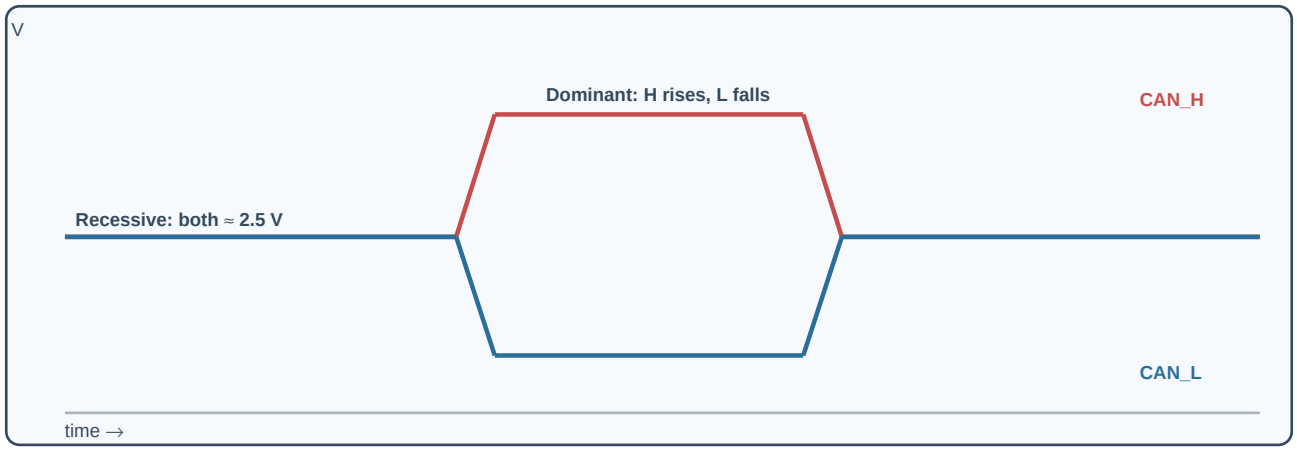
Input bit stream:

```
Original:   0 1 1 1 1 1 0 0 0 0 0 1
After stuff: 0 1 1 1 1 1 0 0 0 0 0 1
             ^           ^
       inserted 0   inserted 1
```

Read the example carefully: the inserted complement appears immediately after the fifth equal bit in a run. The original data stream and the transmitted stream therefore differ by the inserted bit(s). The receiver removes those inserted bits before interpreting the frame.

3.4 Differential signalling — PDF pages 23–24, 28–29

CAN uses two conductors, CAN_H and CAN_L. The receiver interprets their voltage difference rather than treating either wire as an isolated single-ended signal. The PDF's page 24 example shows a recessive state with both lines near 2.5 V and a dominant state with CAN_H near 3.5 V and CAN_L near 1.5 V.



Illustrative waveform redrawn from the voltage levels shown on PDF page 24. The curves are conceptual, not a measured waveform.

Define the differential voltage as Vdiff = V(CAN_H) − V(CAN_L). In the slide's example:

Bus state	CAN_H	CAN_L	Vdiff
Recessive (logic 1)	≈ 2.5 V	≈ 2.5 V	≈ 0 V
Dominant (logic 0)	≈ 3.5 V	≈ 1.5 V	≈ +2 V

The exact voltage limits depend on the physical-layer specification and transceiver. The values above are the illustrative levels drawn in this lecture, not universal threshold limits.

3.5 Why differential signalling helps with EMI — PDF page 29

If electromagnetic interference couples similarly into both conductors, the common-mode disturbance is largely cancelled when the receiver measures the difference between CAN_H and CAN_L. The PDF also mentions optional shielding to further reduce susceptibility and emissions.

3.6 Twisted pair and termination — PDF page 28

The slide describes a twisted pair as a common and inexpensive physical medium. The conductors are CAN_H and CAN_L. It shows a 120 Ω termination resistor at each end of the bus.

The two end terminations belong at the physical ends of the main bus. Do not place a 120 Ω resistor at every node. Kvaser's technical guidance also describes 120 Ω termination at each end for high-speed ISO 11898-2 CAN.

3.7 Physical errors — PDF page 8

The page 8 table lists physical wiring faults. The table below transcribes the visible categories and explains their plain meaning.

Slide error no.	Line(s)	Fault shown on slide	Interpretation
1	CAN_H	Circuit break	High conductor is open.
2	CAN_L	Circuit break	Low conductor is open.
3	CAN_H	Short circuit to Ubat	High conductor is shorted to battery supply.
4	CAN_L	Short circuit to GND	Low conductor is shorted to ground.

Slide error no.	Line(s)	Fault shown on slide	Interpretation
5	CAN_H	Short circuit to GND	High conductor is shorted to ground.
6	CAN_L	Short circuit to Ubat	Low conductor is shorted to battery supply.
7	CAN_H and CAN_L	Short circuit between CAN_H and CAN_L	The differential pair is shorted together.
8	CAN_H and CAN_L	Circuit break of both lines	Both conductors are open.
9	CAN_H and CAN_L	Missing terminating resistors	End termination is absent.

4. CAN transceiver and physical bus

4.1 Transceiver — PDF pages 25–27

The CAN controller works with logic-level transmit and receive signals. The transceiver is the electrical interface to the bus. The page 27 block diagram shows the controller's transmit signal entering the transceiver driver and the bus receiver output returning to the controller.

The page 25 schematic is a TJA1050 example; page 26 shows a Texas Instruments SN65HVD233-HT datasheet excerpt. These are component examples, not mandatory CAN components.

Signal / block	Direction or purpose
TXD / controller transmit	Controller to transceiver; requests a dominant or recessive bus state.
Driver	Creates the electrical CAN_H/CAN_L bus levels.
Receiver	Senses the bus and reconstructs the logic-level receive signal.
RXD / receiver output	Transceiver to CAN controller.

4.2 Fault prevention — PDF page 60

PDF item	Practical meaning
Fiber optics or shielding	Use physical isolation or shielding where the design requires it.
Twisted pair	Improves symmetry of coupled interference.
Ground connection	Provides the required reference/common-mode conditions for the implementation.
Symmetrical differential signals	Preserve balanced signalling on the two conductors.
Less steep edges	Can reduce high-frequency emissions and ringing.
Synchronization	Maintain reliable bit timing between nodes.
Lower baud rate	Reduces timing demands on the physical network.
Shorter bus length	Reduces propagation delay.
Reduction of reflections	Use appropriate topology and termination.

Pages 61–65 are titled Faults / Data Protection, but the extracted slide text contains little or no readable explanatory content beyond page 63's visible phrases: “No Rx → No ack”, “Negative Ack from all Rx” (spelling as shown is unclear), and “Local Rx error”. No additional detailed mechanism is inferred here.

5. Bus logic, arbitration and priority

5.1 Wired-AND — PDF pages 31–32

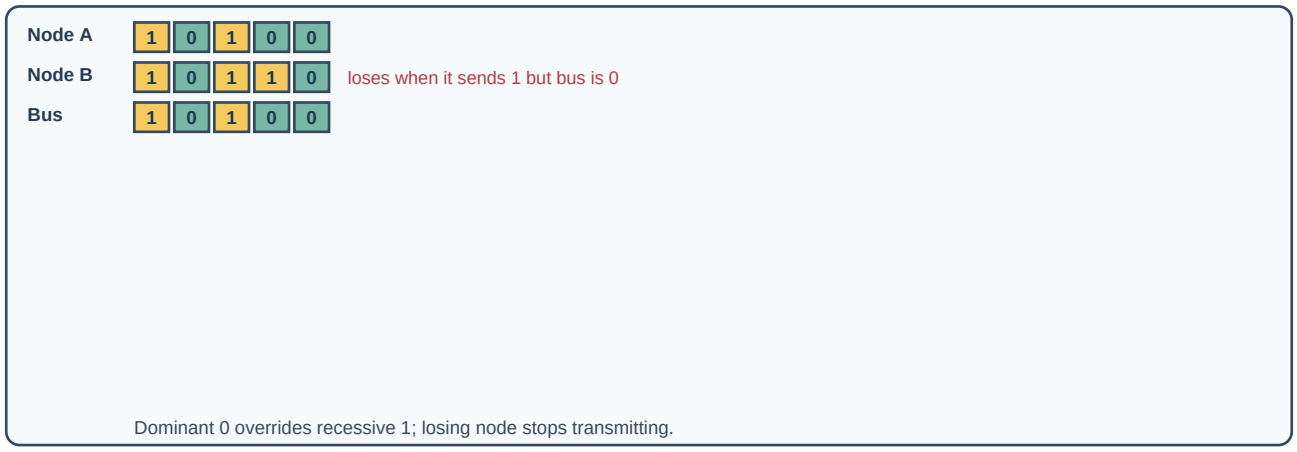
The bus has two logical states: dominant 0 and recessive 1. If every node transmits recessive 1, the bus remains recessive. If any node transmits dominant 0, the bus becomes dominant.

Node A	Node B	Bus result
0	0	0 (dominant)
0	1	0 (dominant)
1	0	0 (dominant)
1	1	1 (recessive)

Dominant 0 overrides recessive 1. This is the electrical/logical foundation of CAN arbitration.

5.2 Non-destructive arbitration — PDF pages 33–36

The lecture labels bus access as CSMA/CD NDA and describes carrier sensing, multiple access and non-destructive arbitration. Nodes that have a frame ready wait for the bus to become idle, then can begin transmitting together. They compare the bus level with the bit they sent.



Conceptual arbitration trace. The bus follows dominant 0 whenever any transmitting node sends 0.

If a node sends recessive 1 but reads dominant 0 on the bus, it has lost arbitration. It stops transmitting and becomes a receiver. The node that continues with the dominant-winning identifier completes its frame. The losing node does not corrupt the winning frame.

5.3 Worked arbitration example

Consider two nodes transmitting identifier bits from most significant bit to least significant bit:

Bit position	Node A	Node B	Bus	Result
1	0	0	0	Both continue
2	1	1	1	Both continue
3	0	0	0	Both continue
4	0	1	0	Node B loses here
5 onward	A continues	B listens	A's bits	A completes frame

The arbitration is bitwise. A lower numerical identifier generally has higher priority because its binary representation tends to place dominant zeros earlier. The actual comparison is made across the arbitration field, not by converting

the entire frame into a decimal priority number.

5.4 Priority — PDF pages 38–39

The slide states that the inverse of the numerical identifier value represents message priority. It gives identifier 0 (0x0) as highest priority and identifier 2047 (0x7FF) as lowest priority for the 11-bit standard identifier range.

11-bit identifier	Relative priority
0x000	Highest
0x123	Higher than 0x456
0x456	Lower than 0x123
0x7FF (2047)	Lowest

The page 39 timeline asks the reader to determine the message sequence for nodes A–D. The image shows identifiers 5, 7, 3, 4 and 6 with requests arriving at different times. The safe method is: (1) let the current frame finish; (2) consider only requests pending when the bus becomes free; (3) select the lowest numerical identifier; (4) repeat. The slide image does not provide a written answer key.

5.5 Bus length and transmission speed — PDF page 37

The slide says arbitration limits bus speed and gives the relation “Maximum speed = 2 × tpd”, defining tpd as propagation delay of the electrical medium. As printed, this expression is dimensionally ambiguous: a speed cannot equal a time. The graph, rather than that printed formula, communicates the intended inverse relationship: longer bus length permits a lower bit rate.

Do not memorize “maximum speed = 2 × tpd” as a valid engineering equation. It is reproduced as printed and flagged as ambiguous. Use the plotted trend and consult a physical-layer design reference for actual timing calculations.

6. CAN protocol scope and higher-layer protocols

6.1 What CAN itself does not specify — PDF page 41

The lecture describes CAN as transporting small packets over a shared communication medium. It explicitly says CAN does not itself define flow control, transport of data larger than an 8-byte message, node addresses or connection establishment.

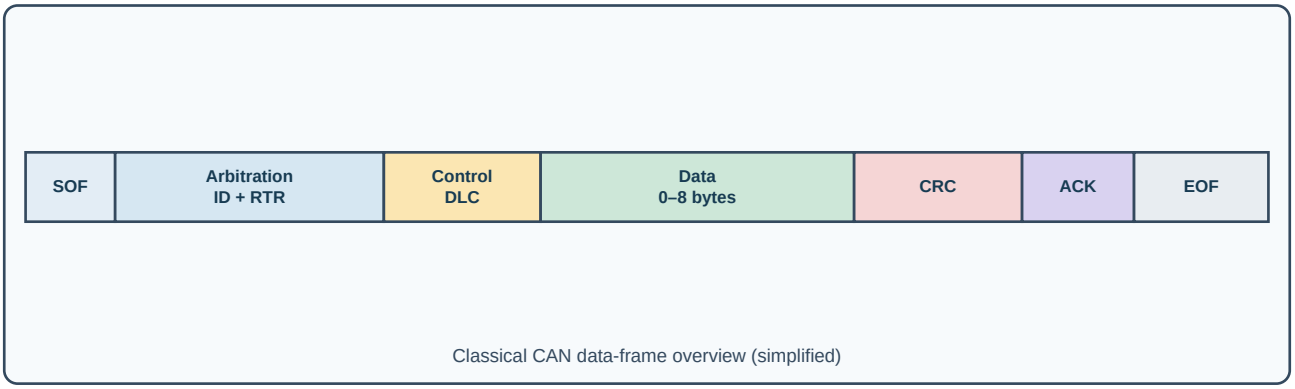
6.2 Higher-layer protocols — PDF page 42

Higher-layer protocols add system-level conventions such as startup procedures, bit-rate configuration, distribution of addresses or message types, message layout and system-level error-handling routines.

Higher-layer protocol named in PDF	Role in this lecture
DeviceNet	Named as an example; no detailed protocol description is provided.
CAN Kingdom	Named as an example; no detailed protocol description is provided.
CANopen	Named as an example; no detailed protocol description is provided.

7. CAN frame structure

7.1 Classical CAN data frame — PDF pages 30, 43, 53–54



Simplified redraw of the frame fields shown on PDF page 43. The exact field layout differs between standard and extended formats.

Field	What it does in the lecture
SOF	Start of Frame; marks the beginning of a frame.
Identifier / arbitration field	Carries the identifier and determines priority during arbitration.
RTR / control bits	Distinguish frame/control information; see standard and extended frame diagrams.
DLC	Data Length Code; indicates the data length.
Data field	0 to 8 bytes in the Classical CAN material shown.
CRC field	15-bit checksum field shown in the lecture.
CRC delimiter	Separates the CRC sequence from the acknowledgement field.
ACK slot and delimiter	Acknowledgement slot and its delimiter.
EOF	End of Frame; shown as 7 bits.
Intermission (ITM)	Shown as 3 bits before the next frame.

The PDF page 43 frame diagram labels the identifier as 11 bits in the standard layout, DLC as 4 bits, data as 0–8 bytes and CRC as 15 bits. Page 53 also describes an extended 29-bit identifier.

7.2 Standard and extended identifiers — PDF pages 51, 54, 59

Format	Identifier width	PDF terminology
Standard / CAN 2.0A	11 bits	Standard CAN
Extended / CAN 2.0B	29 bits	Extended CAN

Page 54 notes that the identifier is not necessarily the semantic name of the data. It is used in arbitration and can be used by receivers to decide which message types they want.

7.3 Data frame — PDF pages 52–54

The Data Frame is described as the predominant message type. It carries zero to eight data bytes. The lecture identifies the arbitration field, data field, CRC field and acknowledgement slot as major components.

An acknowledgement means at least one node received the frame correctly at the CAN protocol level. It does not prove that a particular intended application consumed or acted on the data.

7.4 Remote frame — PDF pages 55–56

A Remote Frame requests a corresponding data frame. The slide says it is marked by a recessive RTR bit and has no data field. The requesting frame uses the identifier of the requested data. The response behaviour is not prescribed by the CAN standard in the way the lecture's example might suggest; controllers can be configured to respond automatically or notify the local CPU.

The PDF highlights a catch: the Remote Frame DLC must be set to the expected response length, or arbitration will not work as intended. It also explicitly warns that a responding node does not start transmitting early to fill the empty remote frame.

7.5 Error frame — PDF page 57

An Error Frame is transmitted when a node detects a fault. The slide describes an Error Flag of six bits of the same value, deliberately violating the bit-stuffing rule, followed by an Error Delimiter of eight recessive bits. Other nodes detect the error and may also signal it. The transmitter automatically attempts retransmission.

The lecture mentions error counters to prevent a node from repeatedly disrupting the bus. Detailed counter thresholds and state transitions are not supplied in these slides.

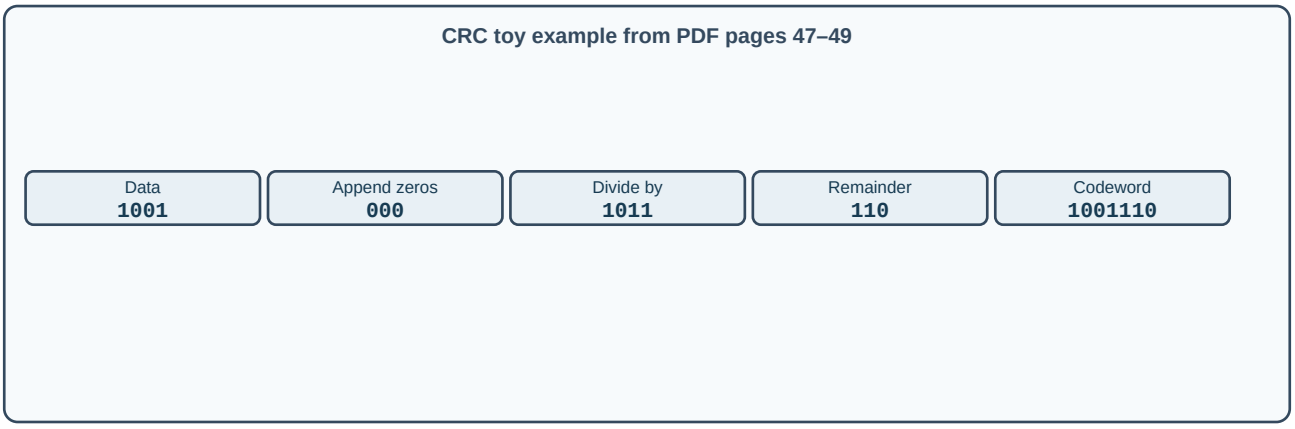
7.6 Overload frame — PDF page 58

The Overload Frame is presented for completeness. The slide says it resembles the Error Frame in format and is transmitted by a node that becomes too busy. It also says it is rarely used and associates generation with the now-obsolete 82526 controller. Treat that as the lecture's historical statement, not a current survey of all controllers.

8. CRC: error detection

8.1 What CRC does — PDF pages 45–49

CRC stands for Cyclic Redundancy Check. The sender treats the data as a polynomial/bit sequence, divides an augmented dataword by a generator, and appends the remainder. The receiver repeats the division on the received codeword. A zero remainder (zero syndrome in the slide's terminology) indicates acceptance in the worked example; a non-zero remainder indicates an error.



CRC workflow and the worked bit pattern shown in the supplied lecture.

8.2 Worked binary division from the PDF

The lecture's worked example uses dataword 1001 and divisor 1011. The divisor has four bits, so three zeros are appended before modulo-2 division. The remainder shown is 110.

```
Dataword:            1001
Augmented dataword:   1001000
Divisor:              1011
Remainder:            110
Transmitted codeword: 1001 110
```

Modulo-2 division uses XOR rather than ordinary subtraction: 1 XOR 1 = 0, 1 XOR 0 = 1 and there is no carry or borrow. When the current leading bit is 0, the divisor is not applied at that position; shift to the next 1.

8.3 Receiver check

Received codeword	Syndrome / remainder	Decision in slide
1001110	000	Accept dataword 1001
1000110	011	Discard dataword

This is the lecture's small illustrative CRC example. Do not confuse its 4-bit generator 1011 with the Classical CAN CRC-15 polynomial. The PDF's CAN frame slide separately labels a 15-bit CRC field.

9. Faults, protection and communication

9.1 Communication example — PDF pages 66–67

The diagram shows five CAN nodes and their transmit (Tx) and receive (Rx) identifier lists. The following page presents a communication matrix. Its central rule is that each message type/identifier has one sender, while multiple nodes may receive it.

When reading the matrix, scan down a message-ID column: the node marked Tx is the sender; nodes marked Rx are receivers. Blank cells mean the slide does not mark that node as a transmitter or receiver for that message.

9.2 Basic CAN controller — PDF page 68

The basic controller diagram shows a CAN protocol controller, acceptance filtering, transmit buffer, receive buffer(s), status/control registers, host interface and bus interface. The slide warns that a cheap/basic controller can overrun the host CPU with messages it does not need.

9.3 Full CAN controller — PDF page 69

The Full CAN slide shows hardware message filters and multiple message objects. It states that hardware filters sort and filter messages without interrupting the CPU, reducing unnecessary host processing.

Basic controller	Full CAN controller (as shown)
Receive path can deliver unnecessary traffic to the host.	Acceptance filtering and message objects screen traffic in hardware.
Higher risk of CPU load from irrelevant messages.	Lower host CPU load for irrelevant messages.

10. CANoe, network descriptions and applications

10.1 CANoe slides — PDF pages 70–75

The lecture uses CANoe to illustrate network simulation, analysis and testing. The slides show ECUs on a high-speed CAN bus, message frames, a trace window, signal interpretation, and a network description.

PDF page	What is visibly shown
70	ABS ECU, Gearbox ECU and Engine ECU on a high-speed CAN bus; example messages.
71	CAN interface and trace/graphics/statistics-style views; an example ID, DLC and data payload.
72	Network description used to interpret data; message name, symbolic name, sender node, raw signal values, physical values and symbolic descriptions.
73	Trace window showing signal names, values and a received CAN frame.
74	Receiving network node, sending network node, signals and a message relationship.

PDF page	What is visibly shown
75	CANoe analysis, simulation and testing around a CAN network description.

The lecture does not provide a step-by-step CANoe lab procedure, a project file or a complete configuration. The slides support understanding the purpose of CANoe, not reproducing a specific simulation setup.

10.2 K-matrix and SAE J1939 — PDF pages 76–80

Page 76 is titled “K matrix / Communication Matrix example”. Page 77 names a CAN (SAE J1939) example involving a Caterpillar 797. Pages 78 and 79 are titled “Caterpillar example”; page 80 contains no readable extracted text. These pages are not detailed enough in the supplied file extraction to justify adding specific J1939 message definitions or vehicle values.

10.3 CANoe download — PDF page 81

The final slide provides a Vector download URL and asks the reader to download and install the trial version. Product availability, licensing and installation requirements may have changed since the lecture was created; consult Vector's current official site before installing.

11. Conceptual simulations and worked exercises

Simulation A — wired-AND bus state

Set each node's transmitted bit, then compute the bus bit. Any dominant 0 forces the bus to 0.

Node A	Node B	Node C	Bus
1	1	1	1
1	0	1	0
0	1	1	0
1	1	0	0
0	0	1	0

Simulation B — arbitration winner

Compare identifiers from the most significant bit. At the first differing bit, a node transmitting 1 while the bus is 0 loses. For standard 11-bit identifiers, the lower numeric identifier has higher priority.

Node	Identifier (binary, 11-bit)	Hex
A	000 0010 0101	0x025
B	000 0010 1100	0x02C
C	000 0011 0000	0x030

Winner: Node A (0x025), because it has the lowest numerical identifier. Node B and C stop transmitting when they detect that the bus is dominant while their own bit is recessive.

Simulation C — mask/filter acceptance

Apply the page 18 rule: filter = 0x00001560, mask = 0x1FFFFFF0. The low four bits are ignored.

Received ID	Accepted?	Reason
0x00001560	Yes	All compared bits match.

Received ID	Accepted?	Reason
0x0000156B	Yes	Only the ignored low four bits differ.
0x0000156F	Yes	Within the 0x1560–0x156F block.
0x00001570	No	A compared bit differs.

Simulation D — bit stuffing

Given a run of five identical bits, insert the opposite bit after the fifth. The receiver removes it. Try both polarities:

```
Input:  00000 1 11111 0
Bus:    000001 1 111110 0
         ^       ^
         stuff 1 after five 0s; stuff 0 after five 1s
```

The spaces above are visual separators. The important rule is the insertion point: immediately after the fifth consecutive equal bit.

Simulation E — CRC

Use the lecture's worked example: dataword 1001, divisor 1011, remainder 110, codeword 1001110. At the receiver, dividing the valid codeword gives remainder 000. Flipping one transmitted bit should generally produce a non-zero syndrome in this example.

12. Exam-focused revision

12.1 Definitions to know

Term	One-line answer
CAN	A shared serial bus for communication among control units.
Broadcast	All nodes can observe a transmission; local filters decide which messages matter.
NRZ	A bit is represented by a level held over the bit time; it does not return to zero between every bit.
Bit stuffing	Insert the opposite bit after five consecutive equal bits; receiver removes it.
Dominant	Logic 0; overrides recessive 1 on the bus.
Recessive	Logic 1; bus remains recessive only when all nodes transmit 1.
Arbitration	Bitwise priority resolution; losing nodes stop without corrupting the winning frame.
Identifier	A field used in arbitration and message classification/filtering; not a destination address.
DLC	Data Length Code.
CRC	Redundancy check used to detect transmission errors.
Transceiver	Electrical interface between CAN controller logic and CAN_H/CAN_L.

12.2 High-yield numerical facts from this PDF

Value	Where it appears	Use it carefully
1 Mbps	Page 9	PDF's stated maximum transfer rate for its stated 40 m example.
40 m / 130 ft	Page 9	Paired with 1 Mbps in the lecture; not a universal limit for every CAN configuration.
8 data bytes	Pages 9, 41, 43, 53	Classical CAN payload limit used in these slides.
120 Ω	Pages 7 and 28	Termination at each physical end in the illustrated bus.
11-bit / 29-bit ID	Pages 43, 51, 53–54	Standard and extended identifier widths.
15-bit CRC	Pages 43, 53	CRC field length shown for the Classical CAN frame.
7-bit EOF; 3-bit intermission	Page 43	As labeled in the frame diagram.

12.3 Likely exam questions

• Why was CAN developed? Explain the wiring problem before CAN and the shared-bus solution.

• Draw and explain the basic CAN node configuration.

• Explain broadcast addressing and local acceptance filtering.

• Explain mask and filter with the 0x1560–0x156F example.

• Explain NRZ encoding and bit stuffing with an example.

• Explain differential signalling and why it helps against EMI.

• Draw CAN_H and CAN_L for recessive and dominant states.

• Explain wired-AND bus behaviour with a truth table.

• Explain non-destructive bitwise arbitration with a two-node example.

• Why does a lower numerical identifier have higher priority?

• Draw and explain the Classical CAN data frame.

• Compare standard and extended CAN identifiers.

• Explain Data, Remote, Error and Overload frames.

• Explain CRC generation and checking using the 1001 / 1011 example.

• Compare basic and Full CAN controllers.

• What is the purpose of a higher-layer protocol? Name the examples in the PDF.

12.4 Common mistakes to avoid

• Calling the CAN identifier a destination address.

• Saying that only one node receives a broadcast frame.

• Reversing dominant and recessive: dominant is 0; recessive is 1 in this lecture.

• Saying a dominant 1 overrides a 0. It is the opposite.

• Assuming a losing arbitration node has corrupted the frame.

• Applying bit stuffing to fields outside the region shown in the lecture.

• Treating the 4-bit toy CRC divisor example as the CAN CRC-15 polynomial.

• Claiming an ACK proves the intended application processed the message.

• Treating every slide heading as if it contained a full explanation. Some pages in this PDF are image-only or contain only a title.

13. Source and visual references

Primary source: the supplied 81-page lecture file, CAN.pdf. All page references in this guide refer to that document.

Selected external technical references

Kvaser — CAN Protocol Tutorial
Broad tutorial covering CAN messages, addressing, physical layer, bit timing and error handling.
https://kvaser.com/course/can-protocol-tutorial/

Kvaser — The CAN Protocol
Concise overview of ISO 11898-1 concepts, differential transmission, arbitration, payload and fault handling.
https://kvaser.com/about-can/the-can-protocol/

Kvaser — CAN Physical Layers
Physical-layer discussion including bus length, bit rate and termination.
https://kvaser.com/lesson/can-physical-layers/

Kvaser — CAN Error Handling
Error detection and error confinement overview.
https://kvaser.com/lesson/can-error-handling/

Vector Informatik — CAN framing visual
Selected external frame-layout visual for standard and extended formats.
https://certification.vector.com/mod/page/view.php?id=49

Texas Instruments — CAN transceiver documentation
Manufacturer reference for CAN physical-layer transceivers and interface design.
https://www.ti.com/interface/can-lin/overview.html

Kvaser — CAN termination guide
Practical discussion of termination and signal integrity.
https://www.kvaser.com/developer-blog/how-to-test-your-can-termination-works-correctly/

Visual policy: diagrams in this guide are either redrawn from the supplied lecture slides or linked to the technical references above. External source figures are attributed by source and URL; the guide does not reproduce unlicensed website screenshots as if they were original lecture material.

Selected external visual links

Vector Informatik — Standard/Extended CAN data-frame diagram
A useful frame-field comparison.
https://certification.vector.com/pluginfile.php/112/mod_page/content/7/CAN_3.2_GRA_StandardExtendedDataFrame_DE.png

CircuitBread — CAN arbitration diagram
A visual explanation of a node losing arbitration.
https://dwma4bz18k1bd.cloudfront.net/tutorials/can-arbitration.png

Embedclogic — CAN physical layer diagram
A twisted-pair physical-layer illustration.
https://embedclogic.com/wp-content/uploads/2019/01/CAN-Physical-Layer.png

14. Scope limitations in the supplied lecture

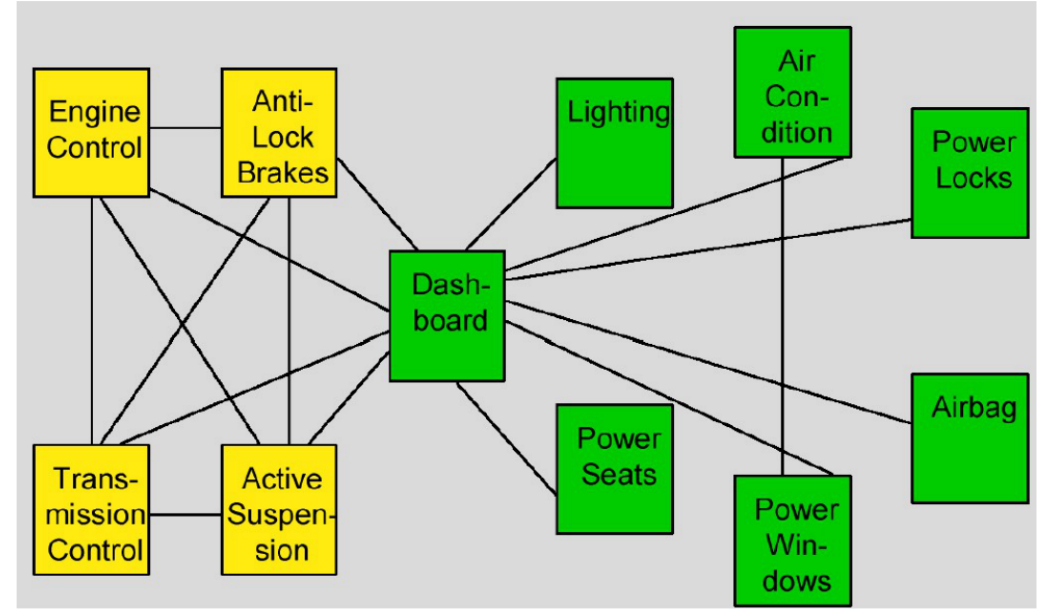
The lecture is a teaching slide deck, not a complete CAN implementation standard. It does not fully specify bit timing register calculations, every frame-control detail, all error-counter thresholds, CAN FD, transport protocols, exact electrical tolerances, or a full CANoe configuration. Those subjects have not been silently added as though they were in the source.

For implementation or hardware design, use the relevant transceiver data sheet and the applicable ISO 11898 documents. Use this guide to understand the concepts and solve questions based on the supplied lecture.

End of guide

Source figure — PDF page 4: Before CAN — point-to-point wiring

The original slide contrasts a dense point-to-point wiring arrangement.



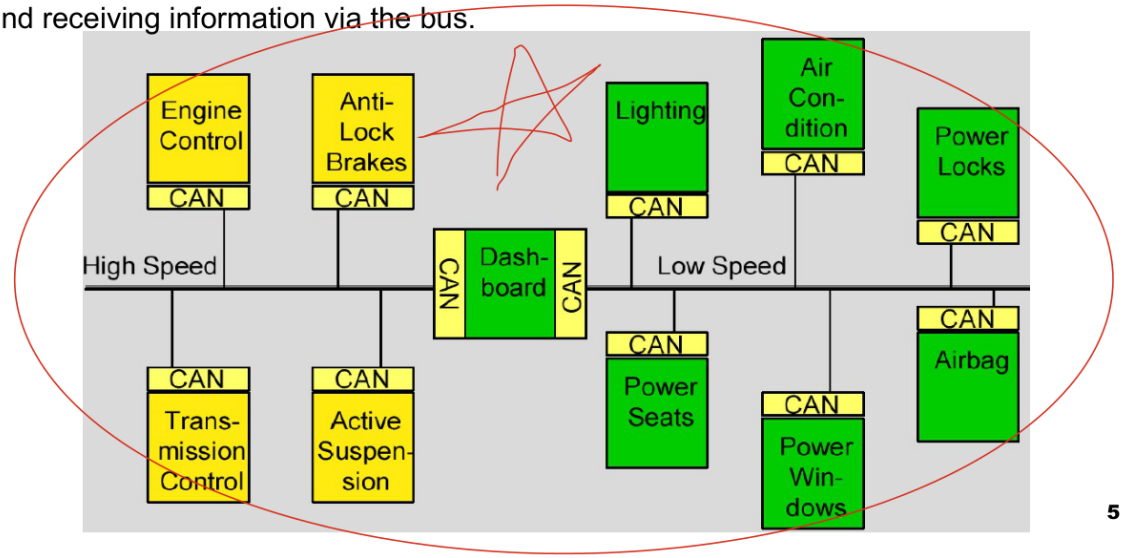
Source: user-supplied CAN.pdf. Original slide image reproduced for study use.

Source figure — PDF page 5: With CAN — shared serial bus

The original slide shows control units attached to a shared CAN bus.

With CAN

The solution to this problem was the connection of the control systems via a serial bus system. This bus had to fulfill some special requirements due to its usage in a vehicle. With the use of CAN, point-to-point wiring is replaced by one serial bus connecting all control systems. This is accomplished by adding some CAN-specific hardware to each control unit that provides the "rules" or the protocol for transmitting and receiving information via the bus.

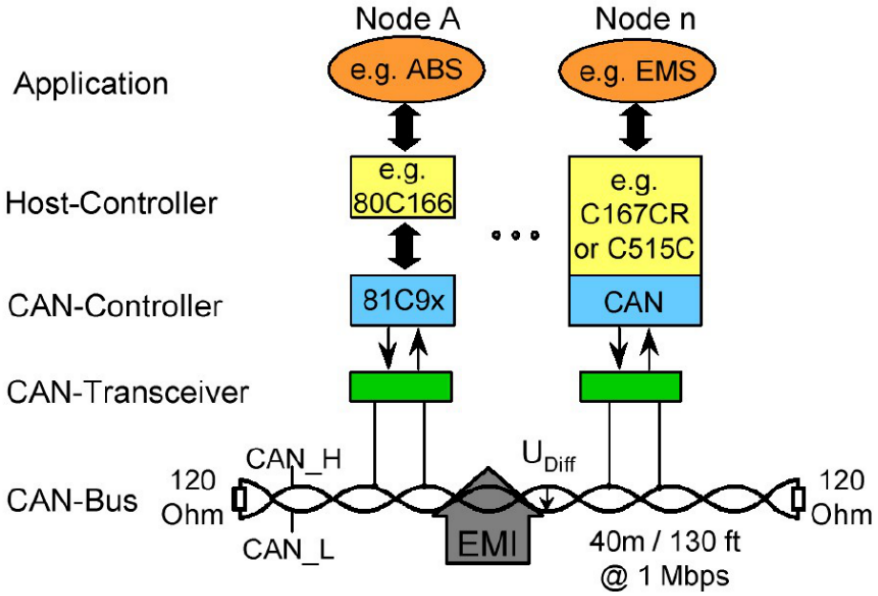


Source: user-supplied CAN.pdf. Original slide image reproduced for study use.

Source figure — PDF page 7: Basic CAN node configuration

Application, host controller, CAN controller, transceiver and terminated bus.

Basic Configuration



7

Source: user-supplied CAN.pdf. Original slide image reproduced for study use.

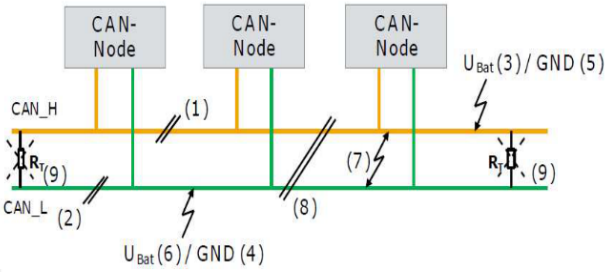
Source figure — PDF page 8: Physical errors

Fault table and bus wiring fault locations.

Physical errors

- Example: High-Speed-CAN

Error Nr.	Line	Error description
1	CAN_H	Circuit break
2	CAN_L	Circuit break
3	CAN_H	Short circuit to U_{bat}
4	CAN_L	Short circuit to GND
5	CAN_H	Short circuit to GND
6	CAN_L	Short circuit to U_{bat}
7	CAN_H, CAN_L	Short circuit between CAN_H and CAN_L
8	CAN_H, CAN_L	Circuit break of CAN_H and CAN_L
9	CAN_H, CAN_L	Missing Terminating Resistors

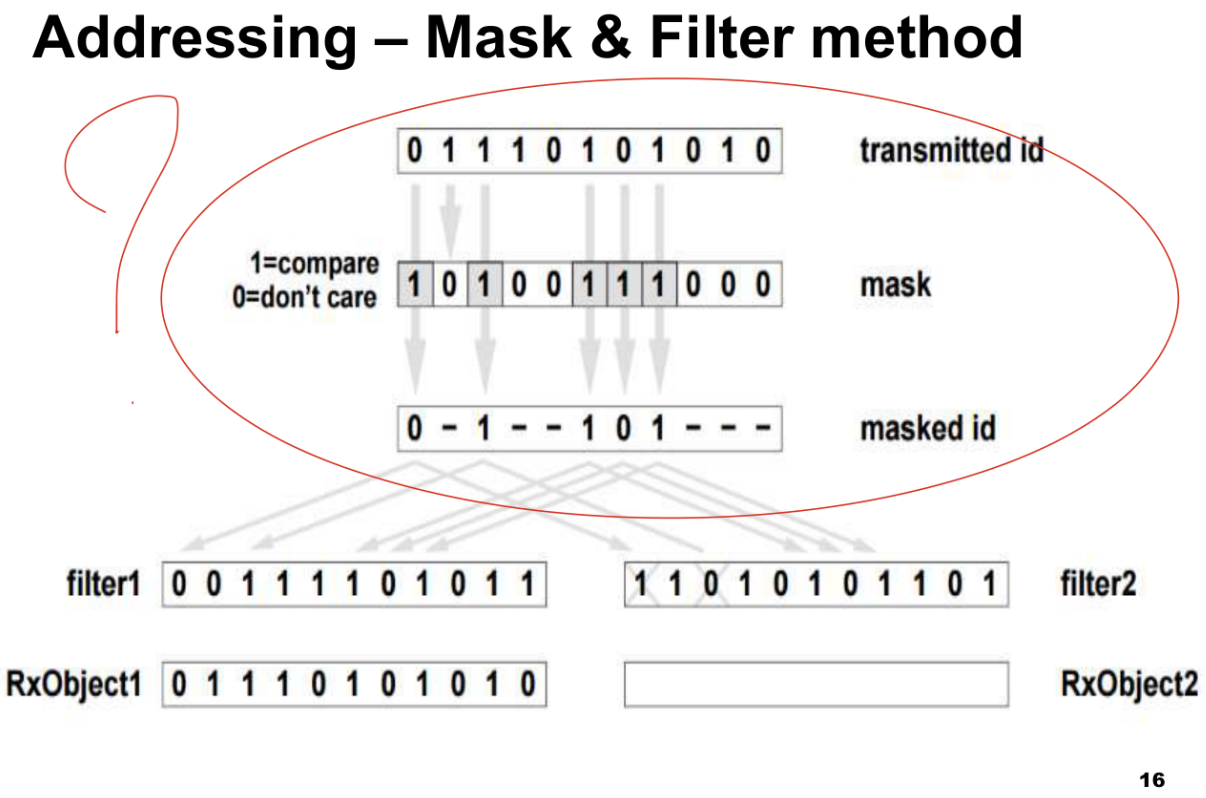


8

Source: user-supplied CAN.pdf. Original slide image reproduced for study use.

Source figure — PDF page 16: Mask and filter method

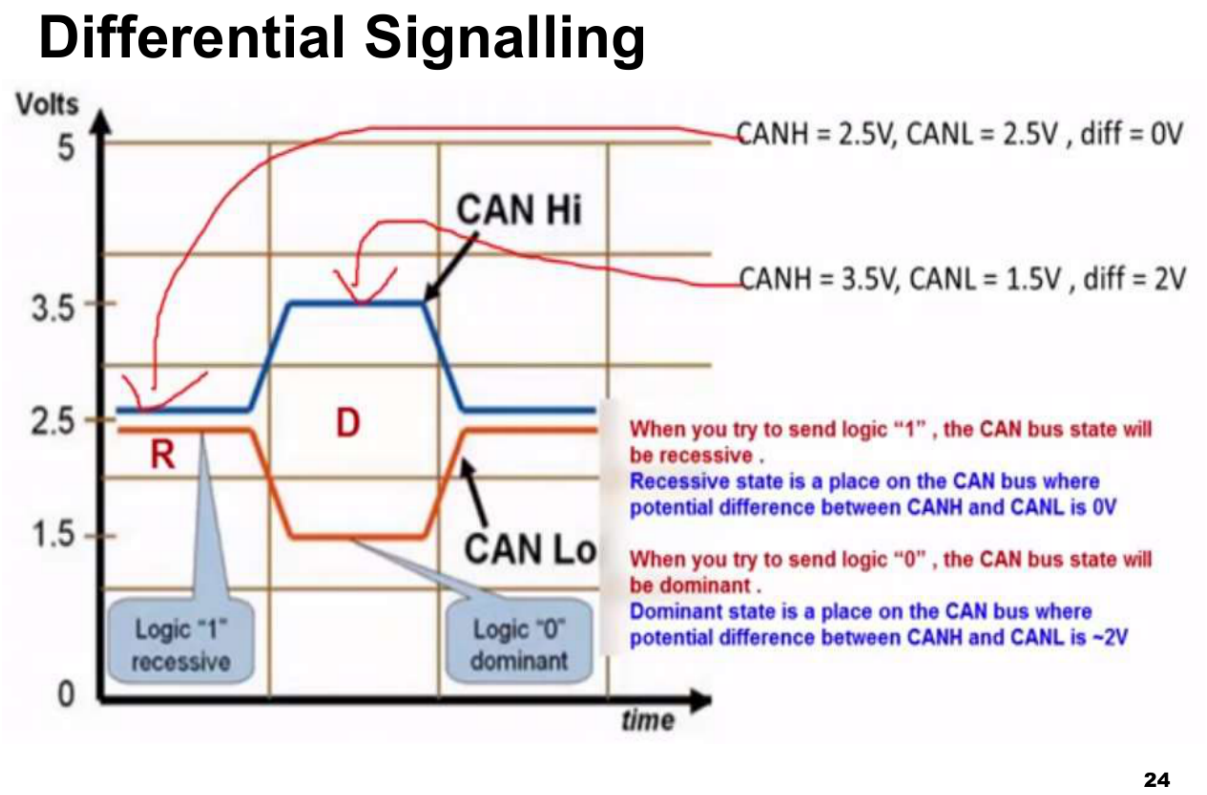
The original bit-mask comparison illustration.



Source: user-supplied CAN.pdf. Original slide image reproduced for study use.

Source figure — PDF page 24: Differential signalling

Recessive and dominant CAN_H/CAN_L voltage levels.

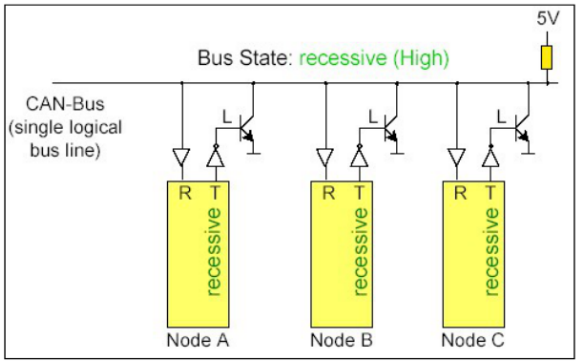


Source: user-supplied CAN.pdf. Original slide image reproduced for study use.

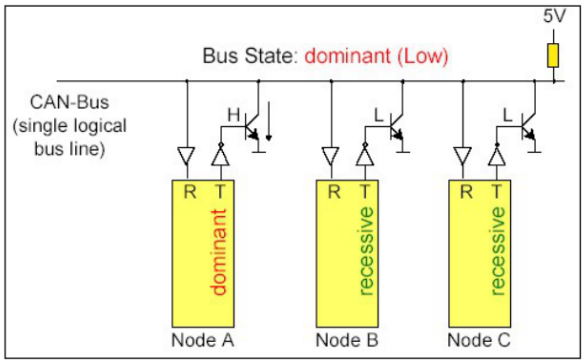
Source figure — PDF page 32: Wired-AND bus behaviour

Dominant and recessive bus states across multiple nodes.

Bus Characteristics – Wired AND



Only if all nodes transmit recessive bits (ones), the Bus is in the recessive state.



If any one node transmits a dominant bit (zero), the bus is in the dominant state.

T is Transmitter, R is receiver. Note nodes can therefore check the line while transmitting. This is important particularly during arbitration.

32

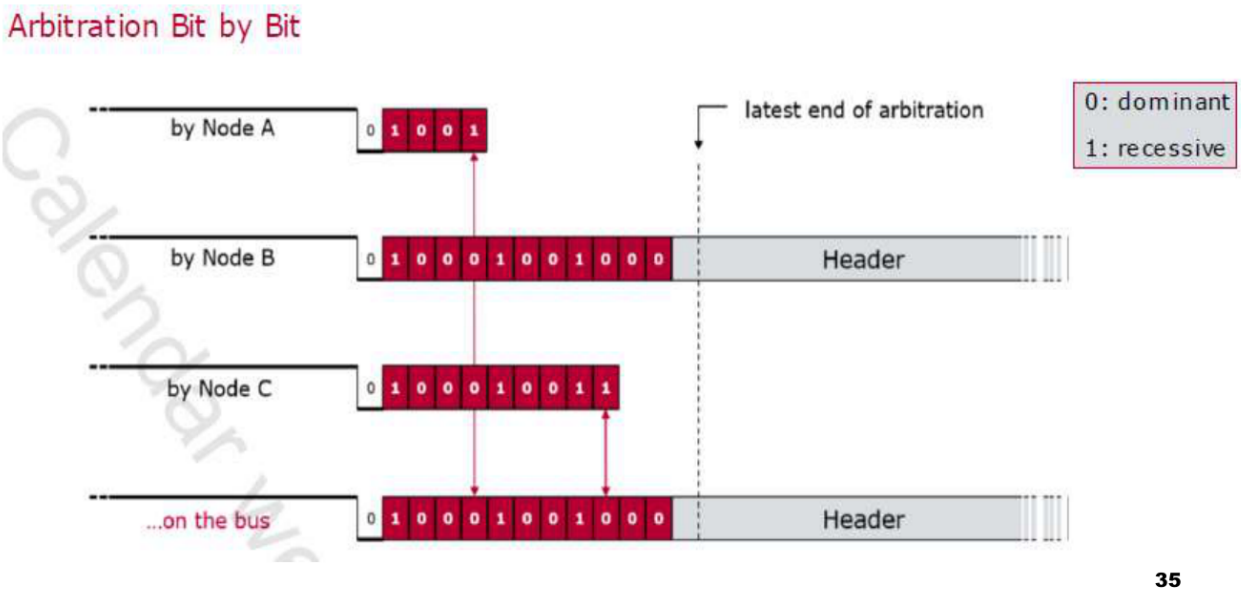
Source: user-supplied CAN.pdf. Original slide image reproduced for study use.

Source figure — PDF page 35: Bitwise arbitration

Arbitration illustrated across competing nodes.

Bus Access and Arbitration – CSMA/CD NDA

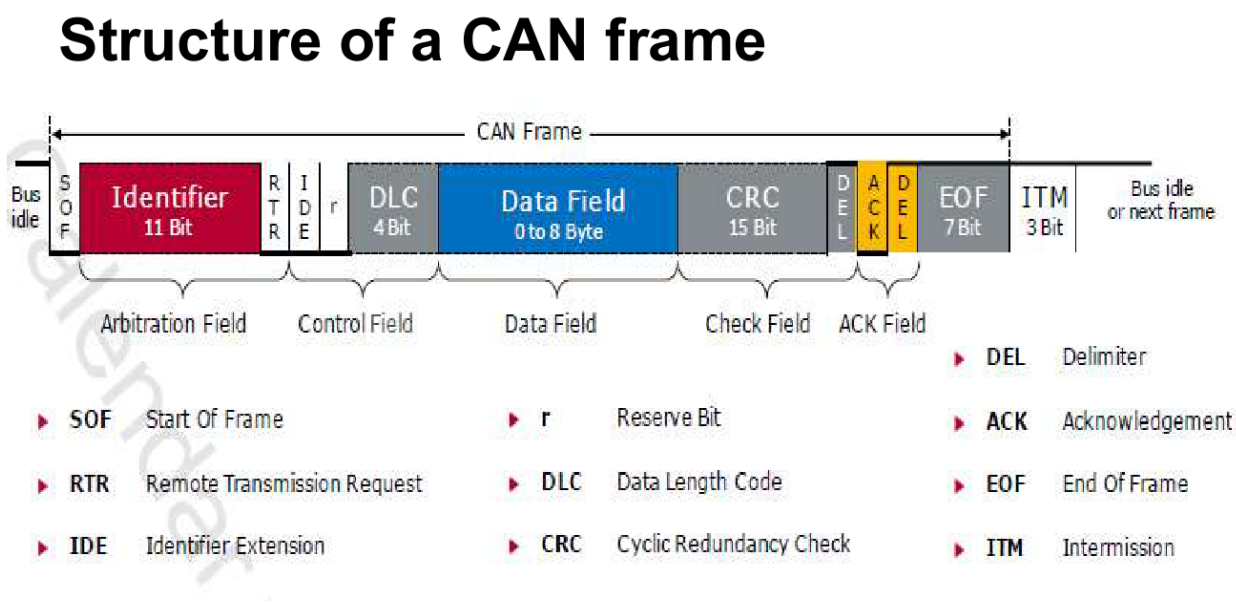
CSMA/CD NDA – Carrier Sense Multiple Access/Collision avoidance by Non Destructive arbitration



Source: user-supplied CAN.pdf. Original slide image reproduced for study use.

Source figure — PDF page 43: Structure of a CAN frame

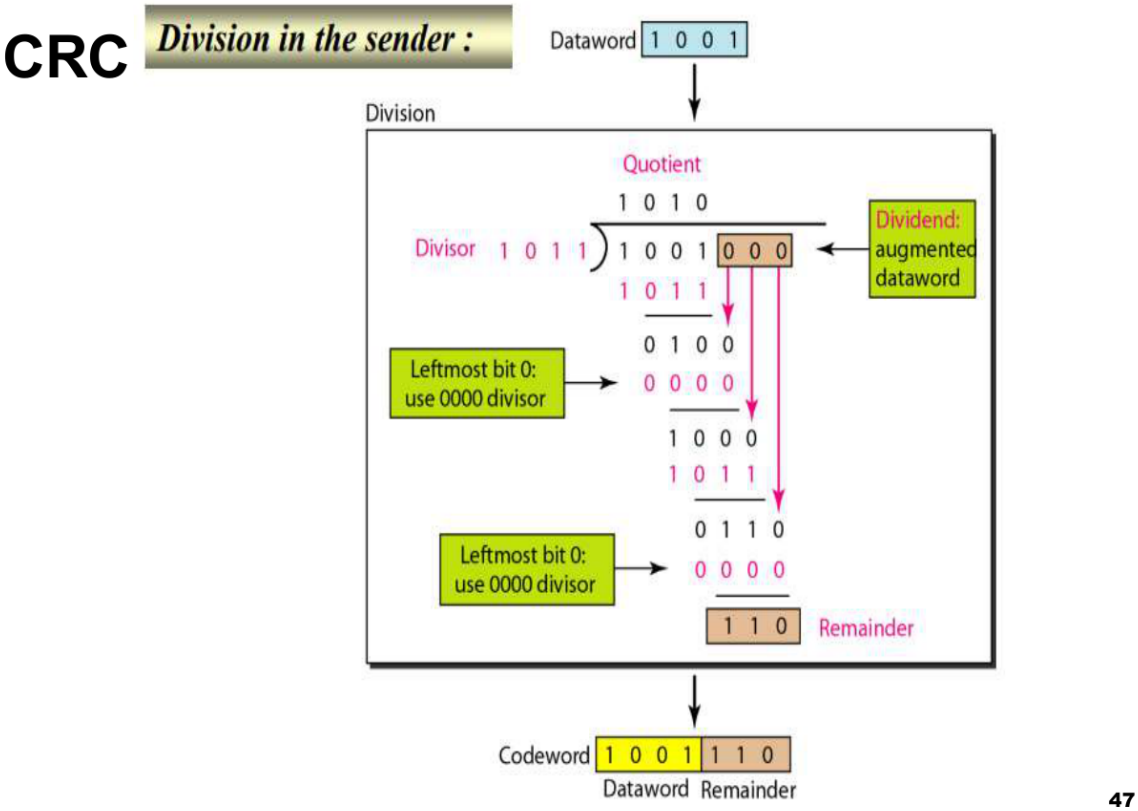
Frame fields and their widths as drawn in the lecture.



Source: user-supplied CAN.pdf. Original slide image reproduced for study use.

Source figure — PDF page 47: CRC worked division

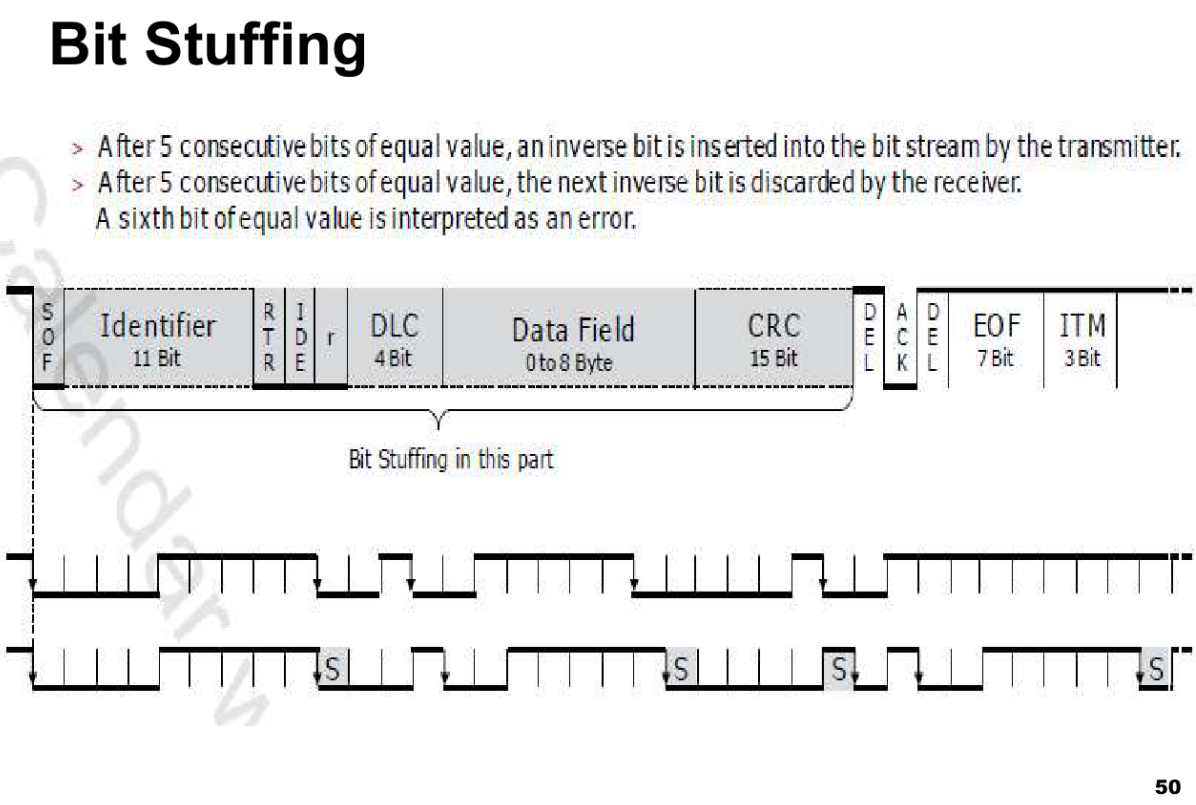
The dataword 1001 divided by generator 1011.



Source: user-supplied CAN.pdf. Original slide image reproduced for study use.

Source figure — PDF page 50: Bit stuffing

Stuffing rule and the portion of the frame where it applies.



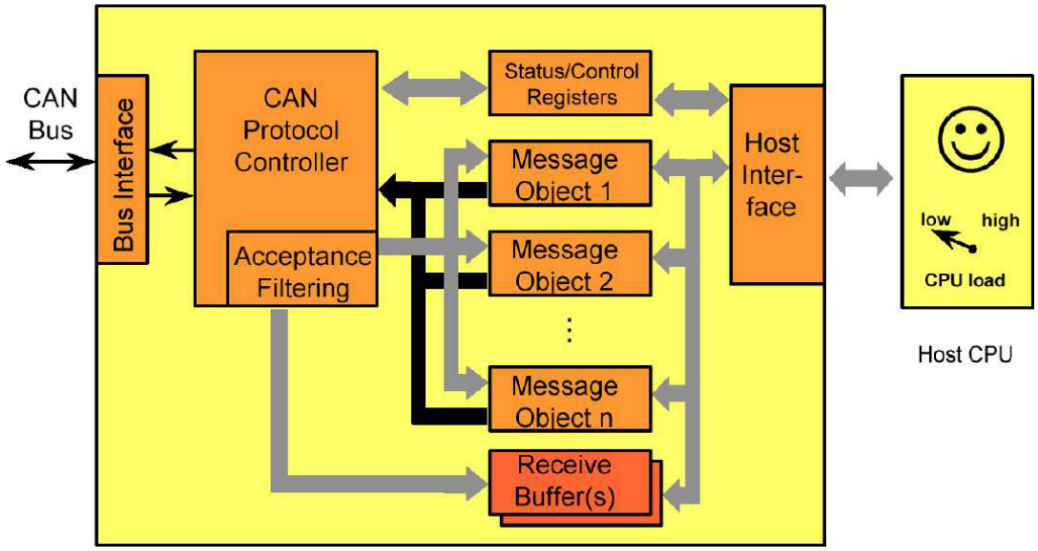
Source: user-supplied CAN.pdf. Original slide image reproduced for study use.

Source figure — PDF page 69: Full CAN controller

Hardware filtering and message objects.

Full CAN Controller

- **Hardware message filters sort & filter messages without interrupting CPU**

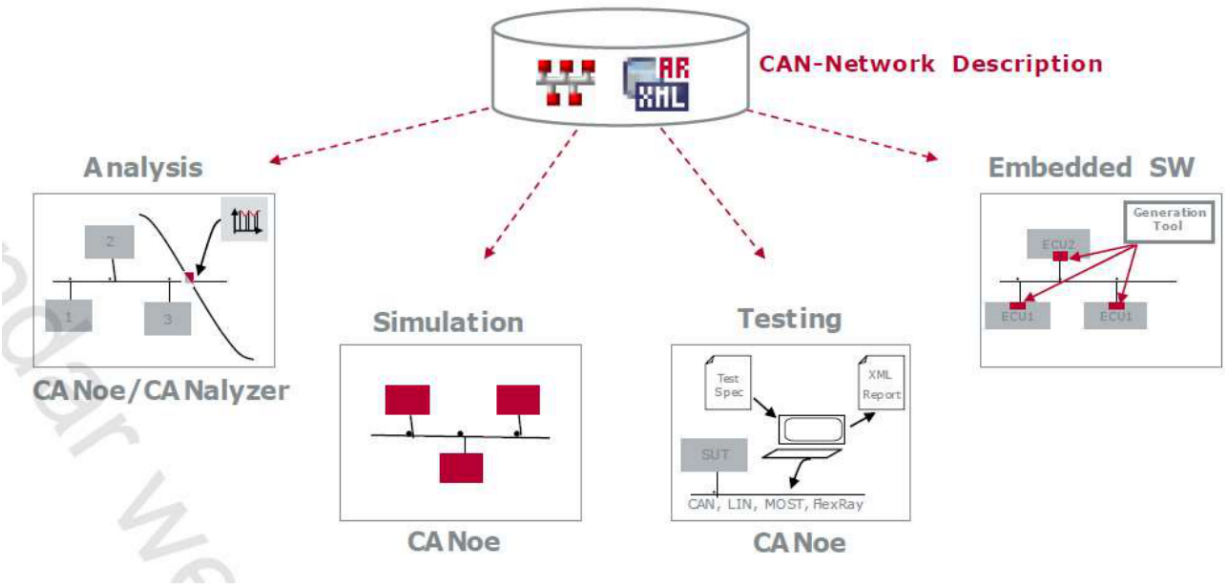


69

Source: user-supplied CAN.pdf. Original slide image reproduced for study use.

Source figure — PDF page 75: CANoe overview

Analysis, simulation and testing around a network description.



Source: user-supplied CAN.pdf. Original slide image reproduced for study use.